High performance computing – Week03- Assignment

Daipayan Bera

1. Tabulate the execution times of each of the individual approaches for computing distance in Python (i.e., run the shared code on your computer, note the times, and tabulate them).

   Python result:

Method	Time
Forloop	2.87 ms ± 569 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
Apply lambda	1.46 ms ± 239 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
Vectorized pandas series	1.3 ms ± 88.9 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
Vectorized Numpy arrays	112 µs ± 7.78 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
Forloop_Cython	2.17 ms ± 143 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
Apply lambda_Cython	1.16 ms ± 39.1 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
Vectorized pandas series_Cython	1.24 ms ± 58.5 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
Vectorized Numpy arrays_Cython	110 µs ± 4.14 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
Forloop_Cython_Dtypes	1.61 ms ± 116 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
Apply lambda_Cython_Dtypes	894 µs ± 34.9 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

2. Next, replicate the for-loop based approach (the first one) and two different ways to make that version more efficient, in R. Profile these three approaches, and tabulate the results.

   R result:

expr	min	lq	mean	median	uq	max	neval	cld
forloop	0.9649	1.00345	1.638869	1.1376	1.55795	15.1773	100	a
apply	1.0652	1.14615	1.511752	1.316	1.6466	4.9038	100	a
vectorized	0.044	0.05355	0.137222	0.08065	0.11745	3.9039	100	b

3. Based on the computational efficiency of implementations in Python and R, which one would you prefer? Based on a consideration of implementation (i.e., designing and implementing the code), which approach would you prefer? Taking both of these (run time and coding time), which approach would you prefer?

   First of all, Python has much more leverage on looping instructions than R, and it's easy to code it, while I faced a bit of difficulty with the assignment of a new vector to the dataframe. But R has variant apply functions. The result of the difference between the for loop and apply functions is pretty much the same for both R and Python. Even the results of vectorized operations are very similar between both languages. But I have observed R profiling is much faster than Python. I had used "%%timeit" profiling for Python and "Microbenchmark" for R. Therefore, for me, Python took less time to code but a long time to profile, whereas R took more time to debug and less time to profile. So, if I were to choose one, I would pick Python, as I am much more comfortable coding and understanding the data structures.

4. Identify and describe one or two other considerations, in addition to these two, in determining which of the two environments – Python or R – is preferable to you.

   Python provides much better version control with virtual environments than R. In Python, virtual environments create completely isolated directories containing their own set of Python and installed packages. But for R, the R interpreter remains shared across all projects.